Effort

Control how many tokens Claude uses when responding with the effort parameter, trading off between response thoroughness and token efficiency.

Copy page

The effort parameter allows you to control how eager Claude is about spending tokens when responding to requests. This gives you the ability to trade off between response thoroughness and token efficiency, all with a single model.

> The effort parameter is currently in beta and only supported by Claude Opus 4.5.
>
> You must include the beta header `effort-2025-11-24` when using this feature.

How effort works

By default, Claude uses maximum effort—spending as many tokens as needed for the best possible outcome. By lowering the effort level, you can instruct Claude to be more conservative with token usage, optimizing for speed and cost while accepting some reduction in capability.

> Setting `effort` to `"high"` produces exactly the same behavior as omitting the `effort` parameter entirely.

The effort parameter affects **all tokens** in the response, including:

- Text responses and explanations
- Tool calls and function arguments
- Extended thinking (when enabled)

This approach has two major advantages:

1. It doesn't require thinking to be enabled in order to use it.
2. It can affect all token spend including tool calls. For example, lower Claude makes fewer tool calls. This gives a much greater degree of control over

Ask Docs

Effort levels

Level	Description	Typical use case
`high`	Maximum capability. Claude uses as many tokens as needed for the best possible outcome. Equivalent to not setting the parameter.	Complex reasoning, difficult coding problems, agentic tasks
`medium`	Balanced approach with moderate token savings.	Agentic tasks that require a balance of speed, cost, and performance
`low`	Most efficient. Significant token savings with some capability reduction.	Simpler tasks that need the best speed and lowest costs, such as subagents

Basic usage

Python

```python
import anthropic

client = anthropic.Anthropic()

response = client.beta.messages.create(
    model="claude-opus-4-5-20251101",
    betas=["effort-2025-11-24"],
    max_tokens=4096,
    messages=[{
        "role": "user",
        "content": "Analyze the trade-offs between microservices and monolithi
    }],
    output_config={
        "effort": "medium"
    }
)

print(response.content[0].text)
```

Ask Docs

- Use **high effort** (the default) when you need Claude's best work—complex reasoning, nuanced analysis, difficult coding problems, or any task where quality is the top priority.
- Use **medium effort** as a balanced option when you want solid performance without the full token expenditure of high effort.
- Use **low effort** when you're optimizing for speed (because Claude answers with fewer tokens) or cost—for example, simple classification tasks, quick lookups, or high-volume use cases where marginal quality improvements don't justify additional latency or spend.

Effort with tool use

When using tools, the effort parameter affects both the explanations around tool calls and the tool calls themselves. Lower effort levels tend to:

- Combine multiple operations into fewer tool calls
- Make fewer tool calls
- Proceed directly to action without preamble
- Use terse confirmation messages after completion

Higher effort levels may:

- Make more tool calls
- Explain the plan before taking action
- Provide detailed summaries of changes
- Include more comprehensive code comments

Effort with extended thinking

The effort parameter works alongside the thinking token budget when extended thinking is enabled. These two controls serve different purposes:

- **Effort parameter**: Controls how Claude spends all tokens—including thinking tokens, text responses, and tool calls
- **Thinking token budget**: Sets a maximum limit on thinking tokens specifically

The effort parameter can be used with or without extended thinking enabled. When both are configured:

Ask Docs

For best performance on complex reasoning tasks, use high effort (the default) with a high thinking token budget. This allows Claude to think thoroughly and provide comprehensive responses.

Best practices

1. **Start with high**: Use lower effort levels to trade off performance for token efficiency.
2. **Use low for speed-sensitive or simple tasks**: When latency matters or tasks are straightforward, low effort can significantly reduce response times and costs.
3. **Test your use case**: The impact of effort levels varies by task type. Evaluate performance on your specific use cases before deploying.
4. **Consider dynamic effort**: Adjust effort based on task complexity. Simple queries may warrant low effort while agentic coding and complex reasoning benefit from high effort.

Economic Futures

Research

News

Responsible Scaling Policy

Security and compliance

Transparency

Help and security

Availability

Status

Support

Discord

Terms and policies

Privacy policy

Responsible disclosure policy

Terms of service: Commercial

Terms of service: Consumer

Usage policy

Ask Docs